Task B2 — Roofline Analysis

Arithmetic intensity and ridge point | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component B, Module 2

At a glance

Question	Answer
Arithmetic intensity	2.5 FLOP/byte (45 MFLOP ÷ 18 MB)
Ridge point	1.33 FLOP/byte (16 GFLOP/s ÷ 12 GB/s)
Classification	Compute-bound: intensity is 1.875× the ridge point
Latency lower bound	2.81 ms (compute) against 1.5 ms (memory)
Optimisation that helps	Cut FLOPs or raise the compute ceiling: INT8 quantisation, pruning, a lighter architecture

1 Inputs and method

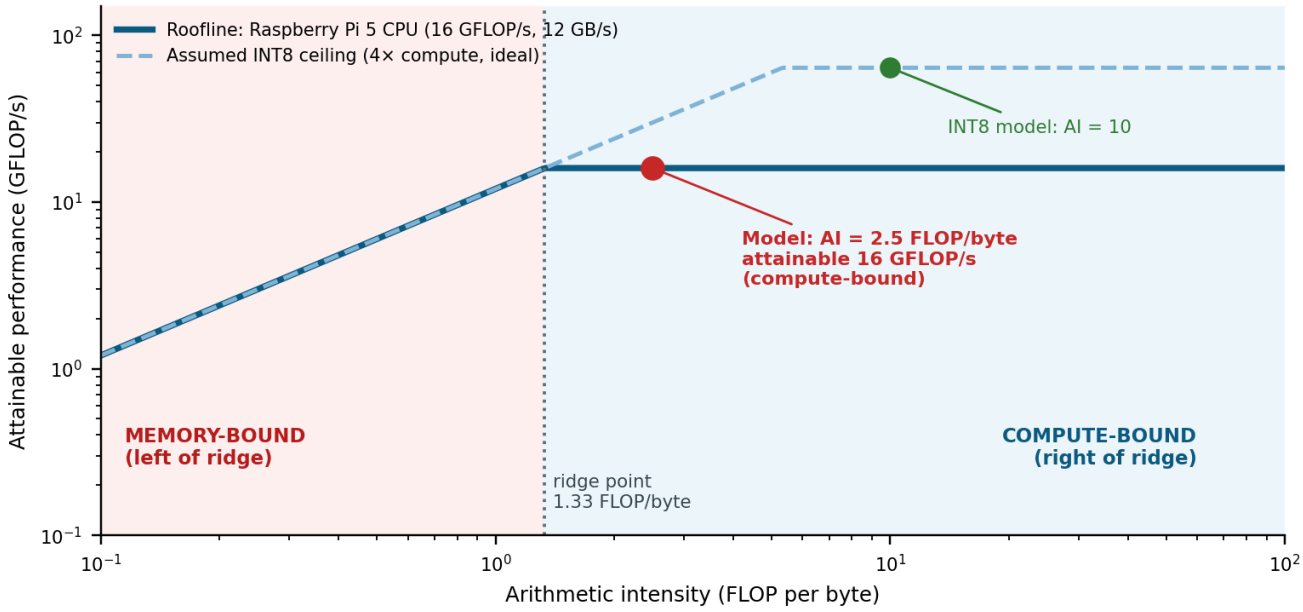
Quantity	Value	Source
Work per inference	≈ 45 MFLOP	Given
Data moved per inference	≈ 18 MB (weights + activations)	Given
Peak compute, Raspberry Pi 5 CPU (NEON)	≈ 16 GFLOP/s	Given
Memory bandwidth, LPDDR4X	≈ 12 GB/s	Given

Roofline relation	Formula
Arithmetic intensity (AI)	FLOPs ÷ bytes moved
Ridge point	Peak compute ÷ memory bandwidth
Attainable performance	min(peak compute, bandwidth × AI)
Latency lower bound	max(FLOPs ÷ peak compute, bytes ÷ bandwidth)

2 Calculation, step by step

Step	Calculation	Result
1 Arithmetic intensity	45×10^6 FLOP ÷ 18×10^6 B	2.5 FLOP/byte
2 Ridge point	16×10^9 FLOP/s ÷ 12×10^9 B/s	1.33 FLOP/byte
3 Compare	2.5 ÷ 1.33	1.875: model sits right of the ridge
4 Bandwidth roof at AI = 2.5	12 GB/s × 2.5	30 GFLOP/s (above the compute ceiling)
5 Attainable performance	min(16, 30)	16 GFLOP/s: the compute ceiling
6 Compute time	$45 \times 10^6 \div 16 \times 10^9$	2.81 ms
7 Memory time	$18 \times 10^6 \div 12 \times 10^9$	1.5 ms
8 Latency lower bound	max(2.81, 1.5)	2.81 ms, set by compute

Classification: the model is compute-bound. Its arithmetic intensity (2.5) is above the ridge point (1.33), so the 12 GB/s memory system could feed 30 GFLOP/s but the CPU can only deliver 16. Compute time (2.81 ms) is 1.875× the memory time (1.5 ms).



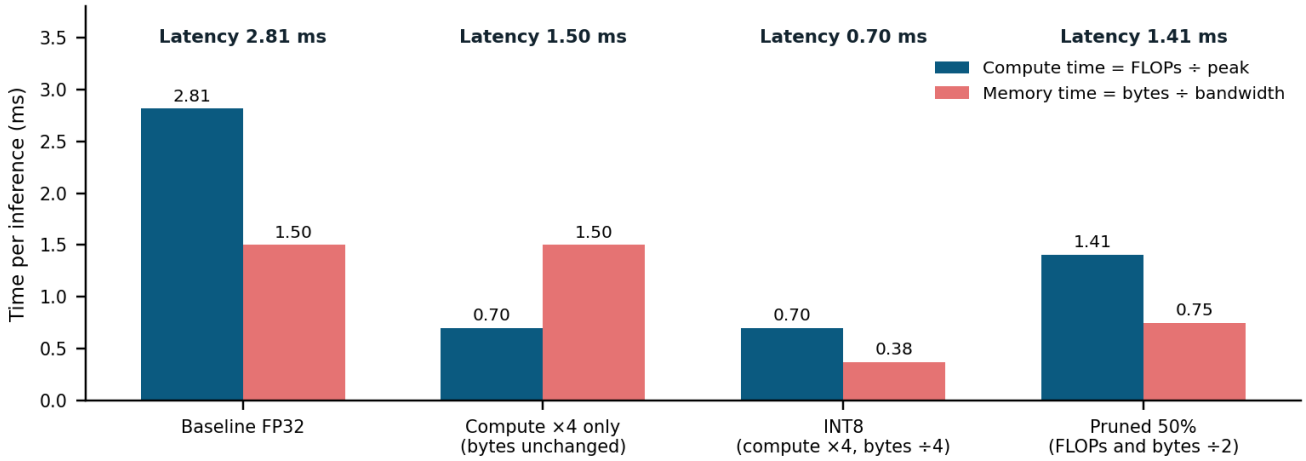
3 Which optimisations the Roofline model favours

Optimisation	Effect on the Roofline	Improves latency?
INT8 quantisation	Cuts bytes about 4× and, with NEON int8 instructions, raises the compute ceiling	**Yes**: attacks both the compute limit and, later, the memory limit
Pruning or a lighter architecture	Cuts FLOPs, so compute time falls in proportion	**Yes**: directly shortens the dominant term
Using all CPU cores, or an NPU such as the Hailo-8L	Raises the compute ceiling	**Yes**, but capped at 1.875× unless bytes also fall (see below)
Operator fusion or tiling to cut memory traffic	Moves the model right, but it is already under the compute roof	**No** gain now: memory time (1.5 ms) is not the bottleneck
Faster or wider RAM	Raises the slanted roof only	**No** gain now: same reason

4 What each optimisation buys: scenario workings

Scenario	AI	Ridge	Compute time	Memory time	Latency	Bound	Gain
Baseline FP32	2.5	1.33	2.81 ms	1.50 ms	2.81 ms	Compute	1.0×
Compute ×4 only (bytes unchanged)	2.5	5.33	0.70 ms	1.50 ms	1.50 ms	Memory	1.875×
INT8 (compute ×4, bytes ÷ 4)	10	5.33	0.70 ms	0.375 ms	0.70 ms	Compute	4.0×
Pruned 50% (FLOPs and bytes ÷ 2)	2.5	1.33	1.41 ms	0.75 ms	1.41 ms	Compute	2.0×

AI = arithmetic intensity in FLOP per byte; ridge = peak compute ÷ bandwidth.



Roofline insight: speeding up compute alone has a ceiling. With bytes unchanged, latency cannot fall below the memory time of 1.5 ms, so the best gain is 2.81 ÷ 1.5 = 1.875× before the model turns memory-bound. INT8 quantisation avoids that ceiling because it shrinks the bytes as well as raising the compute rate.

The 4× INT8 compute gain is an idealised assumption; real NEON gains are usually lower and should be measured. The 50% pruning case assumes bytes fall in proportion to FLOPs.

5 Recommendation and link to earlier tasks

- **Recommended:** INT8 post-training or quantisation-aware training first, then structured pruning if more speed is wanted. Together they cut FLOPs and bytes and keep the model compute-bound at a much shorter latency.
- **Not recommended now:** memory-traffic work (fusion, tiling, faster RAM). The Roofline model shows it would not change latency while the model is compute-bound.
- **Against the 90 s requirement:** the baseline already takes 2.81 ms, which is 0.003% of 90 s and 5.6% of the 0.05 s inference allowance used in Task A1. Optimising is for headroom, battery energy and running more models, not for compliance.
- **Effect on Task B1:** this model (45 MFLOP) is larger than the 10 MFLOP estimate used there. Option 3 (≈ 0.5 GOPS) would take $45 \times 10^6 \div 0.5 \times 10^9 = 0.09$ s, above the 0.05 s allowance but still negligible against 90 s. The Option 1 recommendation stands, and its 2.81 ms CPU time is comfortable.

6 Items to confirm

- Whether 16 GFLOP/s is the single-core or the all-core figure. If it is per core, using four cores could raise the ceiling and the ridge point.
- Whether the 18 MB is FP32 data. The INT8 byte reduction assumes 4 bytes become 1.
- Whether the 45 MFLOP counts a multiply-accumulate as two operations, as assumed here.
- Cache reuse: if part of the 18 MB stays in cache, real traffic is lower, which raises intensity and strengthens the compute-bound result.
- A measured inference time on the Pi 5, to check the 2.81 ms bound.